Patient Module
Frontend Integration & Production Guide

AbrenCare Medical Application
Version 1.0 — August 2026

1. Purpose and Scope

- This guide defines the production-oriented Patient module architecture and frontend API contract.
- The module uses accounts.User as the single source of truth for identity and account information. Patient stores only patient-domain information.

2. Architecture

- User = identity and account lifecycle.
- Patient = patient-specific profile.
- EmergencyContact = emergency contacts owned by a patient.
- MedicalRecord = clinical history; read-only through the patient-facing API.
- MedicalDocument = patient medical files with controlled upload and ownership.
- Account deactivation changes User.is_active to False and preserves patient data.

3. Source of Truth

- Do not duplicate name, email, phone, date of birth, address, city, postal code, or profile picture in Patient.
- These values are exposed from Patient.user.
- The frontend never supplies Patient.user; the backend derives ownership from request.user.

4. Lifecycle

- An authenticated user can create one Patient profile. Unlike Doctor onboarding, patient profile creation does not require administrator approval.
- An inactive User cannot use protected patient functionality.
- Patient profiles are not deleted. Account deactivation preserves historical relationships and medical data.

5. Patient Profile API

- GET /api/patients/me/ — retrieve the authenticated patient's profile.
- POST /api/patients/me/ — create the authenticated user's Patient profile.
- PATCH /api/patients/me/ — update patient-specific fields.
- GET /api/patients/<id>/ — retrieve a patient when authorization permits.
- PATCH /api/patients/<id>/ — update a patient when ownership permits.
- DELETE /api/patients/<id>/ — intentionally unsupported; returns 405.

6. Account Deactivation

- POST /api/patients/me/deactivate/ — deactivates User.is_active.
- The patient profile remains in the database.
- After successful deactivation, the frontend should clear its session and return to the signed-out flow.

7. Emergency Contacts

- GET /api/patients/me/emergency-contacts/ — list contacts.
- POST /api/patients/me/emergency-contacts/ — create a contact.
- GET /api/patients/me/emergency-contacts/<id>/ — retrieve one owned contact.
- PATCH /api/patients/me/emergency-contacts/<id>/ — update one owned contact.
- DELETE /api/patients/me/emergency-contacts/<id>/ — delete one owned contact.
- Use /me/ routes so the frontend does not provide a patient ID to establish ownership.

8. Medical Records

- GET /api/patients/me/medical-records/ — list the authenticated patient's records.
- GET /api/patients/me/medical-records/<id>/ — retrieve one record.
- Patients cannot create, update, or delete clinical records through these endpoints.
- Clinical record changes should use a separate authorized clinical workflow.

9. Medical Documents

- GET /api/patients/me/medical-documents/ — list documents.
- POST /api/patients/me/medical-documents/ — upload a document.
- GET /api/patients/me/medical-documents/<id>/ — retrieve document metadata/file URL.
- DELETE /api/patients/me/medical-documents/<id>/ — delete an owned document.
- The current contract permits PDF, JPG, JPEG, and PNG files up to 10 MB. Production should also validate MIME type, storage policy, and malware scanning.

10. Security

- All protected endpoints require authentication.
- Ownership is enforced server-side using request.user and the related Patient.
- Never trust a frontend-supplied user_id or patient_id for ownership.
- Do not expose unrestricted patient lists to ordinary patients.
- Doctor access to patient clinical data must be based on legitimate clinical relationships, not merely the existence of both profiles.

11. Frontend Integration

- After login, call GET /api/patients/me/ to determine whether the user has a patient profile.
- To create a profile, send only patient-specific fields such as gender and blood_group.
- Treat 401 as an authentication/session issue, 403 as an authorization/account-state issue, 404 as a missing resource, 405 as an intentionally unsupported operation, and 409 as an existing patient profile conflict.
- Use multipart/form-data for medical document uploads.

12. Example Requests

- Create Patient: POST /api/patients/me/ with {"gender":"female","blood_group":"O+"}.
- Create Emergency Contact: POST /api/patients/me/emergency-contacts/ with name, relationship, phone_number, alternative_phone, and is_primary.
- Deactivate Account: POST /api/patients/me/deactivate/ with no patient ID.

13. Recommended Model Rules

- Patient.user should be OneToOneField(User, on_delete=PROTECT, related_name='patient_profile').
- EmergencyContact.patient should have related_name='emergency_contacts'.
- MedicalRecord.patient and MedicalDocument.patient should use PROTECT to preserve clinical ownership/history.
- Consider a conditional unique database constraint allowing zero or one primary emergency contact per patient.

14. Testing Checklist

- Authenticated user can create exactly one Patient profile.
- Unauthenticated access is rejected.
- User cannot create a Patient profile for another user.
- Identity/account fields remain controlled by User.
- Patient deletion is rejected.
- Deactivation changes User.is_active and preserves Patient data.
- Patients can read only their own medical records.
- Patients cannot modify clinical records through the patient API.
- Patients can access only their own medical documents.
- Emergency-contact ownership is enforced.
- Invalid/oversized document uploads are rejected.

15. Appointment Integration

- Appointment should relate Doctor and Patient directly.
- Appointment will become an important authorization boundary for clinical access.
- The Patient model should remain stable while Appointment connects patients to doctors and consultation workflows.
- Do not grant a Doctor unrestricted access to every Patient in the database.

16. Production Roadmap

- Next: finalize Appointment models using the stable Doctor and Patient models.
- Then implement Appointment serializers, APIViews, permissions, URLs, admin, availability validation, conflict prevention, and notifications.
- After that, strengthen clinical workflows with audit logging and controlled medical-record access.

Reference Endpoint Table

Method	Endpoint	Purpose
GET	/api/patients/me/	Current patient profile
POST	/api/patients/me/	Create patient profile
PATCH	/api/patients/me/	Update patient fields
POST	/api/patients/me/deactivate/	Deactivate account
GET	/api/patients/me/emergency-contacts/	List contacts
POST	/api/patients/me/emergency-contacts/	Create contact
PATCH	/api/patients/me/emergency-contacts/{id}/	Update contact
DELETE	/api/patients/me/emergency-contacts/{id}/	Delete contact

Method	Endpoint	Purpose
GET	/api/patients/me/medical-records/	List records
GET	/api/patients/me/medical-records/{id}/	View record
GET	/api/patients/me/medical-documents/	List documents
POST	/api/patients/me/medical-documents/	Upload document
GET	/api/patients/me/medical-documents/{id}/	View document
DELETE	/api/patients/me/medical-documents/{id}/	Delete document

Architecture principle: User is the source of truth for identity/account data; Patient owns patient-domain data; clinical access is explicitly authorized.